

Parametric Geometry Resolver Implementation Report

A deterministic 2D constraint system for bridge templates and drafted components

Audience: Product, CAD-kernel, and bridge-template engineering teams

Date: 7 September 2026

Scope: A general, deterministic 2D relationship resolver and constraint-solver architecture for reusable bridge templates and arbitrary precision-drafted components.

Executive answer

The requested behaviour is feasible, but the correct implementation does not generate a large collection of one-way formulas for every shape. It captures a drawing as primitives, topology, scalar parameters, and declarative geometric constraints. A formula remains useful for a scalar relationship such as $\text{TotalWidth} = \text{CellCount} * \text{ClearSpan} + \dots$; it must not be the mechanism that keeps an inner opening, its walls, and its chamfers connected while a user drags any part of them.

The engine must make three different kinds of knowledge explicit:

1. **Geometry:** points, line segments, circular arcs, circles, and closed loops; their lengths, directions, centers, radii, tangents, and intersections are measurable.
2. **Design relationships:** coincidence, distance, parallelism, perpendicularity, equal length, angle, offset, tangency, symmetry, and containment are equations that the solver keeps true in either edit direction.
3. **Engineering meaning:** labels such as “Clear Span”, “Wall Thickness”, “Top Slab”, and “Web” cannot safely be inferred from coordinates alone. The system can propose them; a template author confirms them once, then a draftsman sees only those curated dimensions.

That division solves the problem in the notebook sketches. When Clear Span changes, the solver keeps the accepted wall-thickness, slab, chamfer, and coincidence constraints satisfied. It moves the required vertices and any attached component instances. It does not uniformly scale the drawing, and it does not ask a draftsman to repair formulas.

The active parametric_formulation branch is a useful prototype, not yet a universal resolver. It has a working formula-acceptance path and passing tests, but its live GAD discovery and adjustment path is based on world-axis bounding boxes, rounded measurements, fixed 45-degree haunch logic, and procedural shape mutation. Those are the specific pieces to replace.

What was verified

This report directly inspected the local repository at commit 3c4956a on parametric_formulation, the supplied PDFs and Markdown research, and current first-party project and standards sources. bun run test completed successfully on 7 September 2026: **41 test files and 171 tests passed**. The Vite configuration warning does not affect the pass result.

The following claims are observations from the current source, not assumptions inherited from the supplied documents.

Area	Direct finding	Consequence
GAD discovery	lib/geometry/gadAssemblyEngine.ts builds candidate regions from shape bounds and uses isGeometricallyContained against minX/maxX/minY/maxY. It rounds clearances, spans, and dimensions before suggestions are created.	The live discovery is not rotation-invariant and treats a polygon's bounding square as its geometry.
Current “solver” path	solveGADAssemblyAdjustment changes rectangle widths, circle radii, polygon radii, and line endpoints through shape-type branches. The loop branch recognizes a haunch from $\text{abs}(\text{abs}(\text{dx}) - \text{abs}(\text{dy})) < 3$, which encodes a 45-degree condition.	It is an assembly-specific procedural adjuster, not a general constraint solve.
Formula acceptance	The Property Inspector dispatches ACCEPT_INFERRED_FORMULA; the reducer stores an expression in the variable model and the normal parametric sync evaluates it.	Accepted inference is directed. A direct geometry edit cannot generally solve the inverse relationship.
Generic model	lib/parametric/model.ts contains extensive shape and magic-ID/name switches for culvert and two-span geometry.	Generalisation currently requires code for each family, the opposite of authoring a new component from geometry.
Good assets to retain	The project includes a formula parser, a constraint graph, local-frame utilities, an LM/SVD path, component types, and tested bridge models.	The target is a staged migration that reuses these pieces behind one canonical model rather than a rewrite.
Coverage boundary	Tests cover single-cell and two-span presets, nested rectangular/circular cases, and 45-degree haunch cases. They do not demonstrate a complete accept-to-bidirectional-solve flow for arbitrary rotated geometry, arbitrary-angle chamfers, or a user-authored non-bridge component.	Passing tests prove current behaviour, not the desired universal capability.

Architecture decision

Adopt a **dual model**:



The DAG only evaluates acyclic scalar definitions and configuration, for example repeat count, allowable bounds, or an explicitly authored engineering calculation. The constraint graph owns spatial relationships. A driving dimension changes a target in the graph; the solver selects a nearby geometry that satisfies the whole graph. This follows established sketcher practice: FreeCAD distinguishes driving dimensional constraints, geometric constraints, and the degrees of freedom remaining in a sketch [FreeCAD Sketcher Workbench](#).

The model must separate **topology** from geometry. A shared corner is one topological vertex referenced by several edges, rather than several numerical endpoints that happen to be close. ISO 10303-42 similarly distinguishes geometric entities from topological entities; use that separation as a design principle, without importing a full STEP boundary-representation implementation into the first 2D release [ISO 10303-42](#).

Canonical primitive model

Every source shape is adapted to a small representation that the resolver understands:

Authoring shape	Resolver representation	Notes
Line or arrow	Segment between two topological vertices	Direction and normal come from endpoints.
Rectangle	Four segments and four vertices	A rectangle creates its own internal parallel/perpendicular facts.
Polygon or polyline	Ordered segments and vertices	Preserve edge ordering, winding, and closed/open state.
Circle	Center point and radius	A full circle is an arc over 360 degrees.
Circular arc	Center, radius, start/end angles, endpoint vertices	Supplies tangent directions.
Ellipse, spline, imported curve	Explicit carrier adapter plus sampled bounds for broad phase	Do not claim full constraint support until a residual/Jacobian implementation exists.

The adapter preserves source-shape IDs for editing and serialization, but relationships refer to stable primitive IDs and topological vertex IDs. Shapes remain a rendering convenience, not the semantic basis of detection.

Relationship vocabulary

The resolver starts with an exact vocabulary whose equations and derivatives are known. It can be extended only by registering a new residual and Jacobian, conformance tests, tolerance behaviour, and serialization rule.

Family	Constraint	Detection measurement	Solver residual concept
Topology	Coincident	Point distance	Same vertex or x/y equality
Linear	Parallel, perpendicular, angle	Cross product, dot product, signed angle	Cross/dot or normalized angle error
Metric	Length, point distance, equal length	Euclidean norm	Squared-distance or length difference
Offset	Point-to-line and parallel-chain offset	Projection on an edge normal	Signed normal distance minus target
Circular	Concentric, radial offset, point-on-circle	Center distance and radius	Center equality or radial error
Tangency	Segment-circle, circle-circle, arc variants	Center-to-carrier distance	Distance minus radius / sum / difference
Incidence	Point-on-line, point-on-arc, collinear	Cross product or radial/angular check	Carrier distance and range condition
Symmetry	Point/edge symmetry about an axis	Reflection matrix error	Reflected coordinate difference
Reference	Fixed point, fixed angle, construction axis	Explicit author action	Coordinate or direction target

Containment, repeated spacing, “wall”, “clear span”, and “cell” are **not** primitive constraints. They are higher-level candidate patterns built from the vocabulary. A bridge template may define a wall as two offset edge chains and a cell as a bounded void between them. A railing template may define a repeated post pattern. Neither case adds bridge-specific math to the solver.

Vector and matrix mathematics

Vectors and matrices are the correct basis because the measurements below are invariant if the complete component is rotated or translated.

For a segment from point A to B, let the direction be $d = B - A$, its unit direction $u = d / |d|$, and its unit normal $n = (-u_y, u_x)$.

- **Parallel:** the normalized cross product $\text{cross}(d1, d2) / (|d1||d2|)$ is zero for parallel edges.
- **Perpendicular:** the normalized dot product $\text{dot}(d1, d2) / (|d1||d2|)$ is zero for perpendicular edges.
- **Wall thickness:** the signed offset from a point P to a reference edge is $\text{dot}(P - A, n)$. Two truly parallel boundary edges keep the same offset at both sampled ends.
- **Angle:** $\text{atan2}(\text{cross}(d1, d2), \text{dot}(d1, d2))$ gives a signed relative angle. It replaces the current fixed 45-degree test; 30, 45, and 60 degree chamfers are all represented by their measured or authored angle.
- **Concentric circles:** center separation is $|C1 - C2|$; radial thickness is $|r1 - r2|$.
- **Segment-circle tangent:** the perpendicular distance from center C to the segment's supporting line equals the radius, subject to the projected contact point lying within the segment range.
- **Symmetry:** reflecting point P about an axis through O with unit normal n uses $P' = O + (I - 2nn^T)(P - O)$. P and Q are symmetric when $|P' - Q|$ is within tolerance.

Use 3 by 3 homogeneous affine matrices for component placement, not predicate detection. A child component transform is composed from the parent frame, parent port frame, user offset, and inverse child-port frame. That keeps a cell, pipe, post, or custom component reusable after rotation and placement:

```
WorldTransform(child) = WorldTransform(parent)
    * ParentPortFrame
    * UserOffsetRotation
    * inverse(ChildPortFrame)
```

Candidate inference and semantic confirmation

Inference must be conservative. It produces suggestions, never silent engineering intent.

```
normalise shapes
-> weld topological vertices
-> index primitive carriers
-> retrieve nearby candidate pairs
-> measure vocabulary predicates
-> cluster equal measurements
-> reject redundant or conflicting rows
-> rank and show author suggestions
-> author confirms semantic role and driving choices
-> create constraints and parameters
```

The broad phase uses an R-tree or similar spatial index around the edited primitives. The narrow phase only runs applicable predicates, so it does not compare every edge to every other edge. The existing detectGADAssemblies nested scan is acceptable as prototype code but must not become the production path for large drawings.

Each candidate includes primitive references, a measured value and units, tolerance/deviation, confidence, provenance, and its proposed residual type. Before display it must pass a Jacobian rank test against accepted constraints:

- An independent row may be offered.
- A dependent, already satisfied row is redundant and is hidden.
- A dependent, unsatisfied row conflicts and is rejected with an explanation.

Cluster candidates before presentation. Four equal chamfer measurements become one shared Chamfer Size candidate with four occurrences, not four unrelated formulas. A rejection is remembered for the candidate's primitive set and session so the UI does not repeatedly suggest the same accidental alignment.

The author then gives a candidate a semantic name and role. For example, the system may propose “parallel offset 350 mm on these four segments.” The author decides whether that is WallThickness, an incidental gap, or should be ignored. This is the only safe way to build a bridge vocabulary without hardcoding bridge knowledge into every geometry detector.

Solver contract

The numerical core receives a connected subgraph, current state, accepted constraints, and an edit request. It returns a structured status, never only a new array of coordinates.

Inputs

- Stable entity, parameter, constraint, and component-instance IDs.
- Model-space units, tolerances, and numeric policy.
- Current solved state as the initial guess, preserving continuity and orientation branch where possible.
- Constraint strengths: fixed, driving, hard, soft inference, reference/measurement, and temporary drag.
- Solve mode: interactivePreview or commit.
- Budget: algorithm preference, time/iteration limit, residual tolerance, and allowed fallback.

Outputs

- Updated geometry and parameter values.
- Convergence status, maximum residual, weighted residual norm, iteration count, algorithm, and time.
- Degree-of-freedom result per connected component: under-constrained, well-constrained, redundant, conflicting, malformed, or non-convergent.
- A minimal explanation mapped from constraints back to semantic names such as Clear Span and Wall Thickness.
- Provenance sufficient to reproduce the solve with the same input graph and numeric policy.

Do not hide interactive dragging inside permanent dimensions. A drag becomes a temporary high-priority point/edge target while the pointer moves. On release, the user may convert its measured result into a driving constraint or discard it. The [@salusoft89/planegcs](#) README documents a WebAssembly/browser and Node wrapper around FreeCAD's 2D solver, including driving, non-driving, and temporary constraints plus DogLeg, Levenberg-Marquardt, BFGS, and SQP options [Planegcs README](#). It is a credible adapter to spike behind this contract, not a replacement for the template, semantic, topology, or UI layers.

Use the existing hand-derived culvert residuals where they are correct. Do not replace validated domain residuals just because a generic solver is adopted. Add an adapter that can route an eligible connected component to PlaneGCS; retain the project solver for domain-specific residuals until benchmarked. Confirm the released package's LGPL obligations with legal review before distribution: current repository, npm, and upstream metadata are not identically worded.

GEOM RP 2 internal protocol

GEOM-RP/2 is an internal interoperability and authoring protocol. It is **not** an IEEE, ISO, or STEP standard. Its requirements use MUST, SHOULD, and MAY in the conventional RFC 2119 sense [RFC 2119](#). ISO STEP standards can inform a later exchange mapping: ISO 10303-108 covers constraints and design intent in 2D sketches while leaving solution method and UI behaviour application-defined [ISO 10303-108](#), and ISO/TS 10303-1788/1789 provide useful explicit-constraint and dimension terminology.

4. **Identity and units.** A conforming document MUST use stable IDs for primitives, topological vertices, constraints, parameters, ports, and component instances. Every dimensional value MUST state its unit. Documents MUST NOT use pixels or viewport zoom as model tolerance.
5. **Topology.** A conforming resolver MUST store intended shared endpoints as a topological identity or an explicit coincidence constraint. It MUST preserve ordered loop membership and orientation.
6. **Geometry.** The core Level 1 primitive set is point, segment, circle, arc, and open/closed polyline. Unsupported curves MAY be preserved as carriers but MUST NOT advertise unsupported editable constraints.
7. **Constraint schema.** A constraint MUST contain its type, referenced entities, target value or parameter, strength, provenance, state, and current diagnostic result. A scalar formula MUST remain separate from a geometric constraint.
8. **Detection.** Automatic detection MUST operate on primitives and model-space tolerances. It MUST NOT use a page-axis bounding box or a hardcoded drawing angle as proof of an invariant.
9. **Inference lifecycle.** An inferred relationship MUST remain a candidate until accepted by an author or an explicitly enabled auto-constraint policy. It MUST carry a measured deviation and confidence. Rejected candidates SHOULD be suppressed for the same session and primitive set.
10. **Admissibility.** Before accepting a candidate, the resolver MUST identify it as independent, redundant, or conflicting against the current connected constraint graph. A conflicting candidate MUST NOT be silently inserted.
11. **Solve request.** An interactive request MUST declare temporary constraints separately from persistent driving constraints. The solve result MUST report convergence, residual, diagnosis, and changes.
12. **Components and ports.** Templates MUST store internal geometry in local coordinates and expose named ports. Instances MAY attach by compatible point, edge, or axis ports through affine composition.
13. **Drafting workflow.** A draftsman MUST be able to edit an exposed driving dimension by badge or grip without reading formulas. A derived dimension MUST be visibly locked or explain the driving dimensions that control it.
14. **Conformance.** Level 1 covers coincidence, distance/length, parallel, perpendicular, point-on-line, and fixed entities. Level 2 adds offset, angle, concentricity, arc/circle tangency, and symmetry. Level 3 adds component ports, repeat arrays, curve adapters, and verified exchange mapping.

Template and port model

Replace the current parallel template formats with one versioned TemplateDefinition. It owns parameters, semantic roles, local primitives, constraints, ports, repeat rules, accepted inference provenance, and standards metadata. A component instance provides driving overrides and one optional port attachment.

```
type ParameterRole = 'DRIVING' | 'DERIVED' | 'FIXED' | 'MEASURED';
```

```
interface Port {  
  id: string;  
  kind: 'point' | 'edge' | 'axis';  
  localFrame: { origin: PointExpr; angle: ScalarExpr };  
  direction?: 'in' | 'out' | 'bidirectional';  
}
```

```
interface ComponentInstance {  
  id: string;  
  templateId: string;
```

```
overrides: Record<string, ScalarExpr>;
attachment?: {
  parentInstanceId: string;
  parentPortId: string;
  ownPortId: string;
  along?: ScalarExpr;
  normal?: ScalarExpr;
  rotation?: ScalarExpr;
};
}
```

This is general because it describes attachment rather than bridge shapes. A BoxCell can attach its leftWallAxis to another cell's rightWallAxis. A RailingRun can place Post instances along a deck-edge port. A custom user component uses the same format once the author exposes ports.

User experience

Draftsmen work with grips and dimensions. Authors work with inference and engineering intent.

User	Visible controls	Hidden implementation detail
Draftsman	Exposed parameters, dimensional badges, AutoCAD-style move/stretch grips, solver status, plain-language conflicts	Formula strings, candidate ranking, Jacobian, matching, ports unless the template exposes them
Template author	Candidate list with source geometry, measured value, semantic rename, constraint roles, ports, repeats, limits, standards metadata	Raw solver vectors unless diagnostic mode is opened
Kernel developer	Primitive adapters, residual registry, Jacobian tests, diagnostic graphs, benchmarks	No semantic assumption inferred from code path alone

The notebook example becomes straightforward. The author accepts two parallel-offset chains as Wall Thickness, top and bottom offsets as Slab Thickness, and the inner opening's horizontal dimension as Clear Span. The solver has enough information to keep these values. If the author marks outer width derived, increasing Clear Span moves the right-side geometry and grows overall width. If outer width is driving, the free straight portions shrink instead. The system presents that choice as driving versus derived when authoring; it never guesses it from one static drawing.

Staged implementation plan

The stages are ordered so every step delivers a testable improvement and none claims universal support before its residuals and UI are in place.

Phase 0 source of truth and safety

Create the canonical schema and an adapter facade. Freeze the current procedural GAD adjuster behind a legacy interface. Add fixture drawings for rotated nested rectangles, arbitrary-angle chamfers, concentric and eccentric circles, two-cell culvert, and a non-bridge railing run. Capture current outputs as regression fixtures where behaviour is intentional.

Exit criteria: A single document model can represent the fixtures without shape-ID switches; no geometry is changed in this phase.

Phase 1 primitive and topology foundation

Create primitive adapters, stable IDs, model-unit tolerance policy, vertex welding, loop orientation, and a spatial index. Rectangles and polygons must lower into shared segment/vertex graphs. Preserve source editing IDs and only weld under a declared tolerance.

Exit criteria: Rotating any test fixture does not change its primitive graph or pairwise vector measurements. Disconnected components have separate topological and solve partitions.

Phase 2 Level 1 predicates and candidates

Implement coincidence, segment length/distance, parallel, perpendicular, point-on-line, and fixed constraints. Replace axis-aligned discovery in the candidate path with vector calculations. Keep current formula cards as a temporary UI adapter, but emit typed constraint candidates rather than formulas.

Exit criteria: A rotated nested rectangle produces the same offset candidates as the unrotated version; arbitrary polygons use their actual edges rather than bounding boxes.

Phase 3 solver orchestration and direct dimensions

Make dimension badges create/update a driving constraint, then re-solve the affected connected component. Wire the existing admissibility calculation into candidate acceptance. Replace global DOF estimates with connected-component diagnosis and a real matching/decomposition implementation. Show draftsmen a neutral “free”, green “fully defined”, or red conflict status with semantic names.

Exit criteria: Editing a committed Clear Span badge re-solves geometry bidirectionally; a deliberately redundant and a deliberately conflicting dimension produce distinct results.

Phase 4 Level 2 circular and angular relationships

Implement signed angle, general chamfer, parallel offset, concentricity, radial offset, point-on-circle/arc, segment-circle tangency, circle-circle tangency, and symmetry. Delete the fixed 45-degree inference rule after its measured-angle replacement passes all regression tests.

Exit criteria: 30, 45, and 60 degree chamfer fixtures retain their authored angle; circles remain concentric/tangent after a driving edit and a component rotation.

Phase 5 component templates and ports

Migrate BUILTIN_TEMPLATES, orphan RCC generator data, and reusable preset definitions into the canonical schema. Implement local-frame port composition and repeat rules. Add template validation for unbound ports, expression cycles, duplicated driving dimensions, range limits, and unresolved solver errors.

Exit criteria: A two-cell culvert and a railing/post fixture are both composed from ports and repeats, without geometry-family-specific placement code.

Phase 6 solver adapter and performance

Build a thin PlaneGCS adapter that maps only supported primitive/constraint types. Benchmark it against the local LM/SVD implementation and the single-cell/two-span culvert residuals. Partition solves by connected component; keep temporary drag constraints in preview mode. Add deterministic fixture replay and numeric tolerance reporting.

Exit criteria: The selected solver meets interaction and convergence thresholds on the published benchmark suite. Unsupported constraints remain routed to the local residual registry or reported clearly.

Phase 7 authoring release gate

Complete candidate clustering, rejection memory, semantic vocabulary curation, template versioning, conformance report generation, and export/import validation. Add end-to-end browser tests for author acceptance, draftsperson badge edit, direct drag, conflict recovery, port attachment, repeat count change, undo/redo, and serialization.

Exit criteria: A template author can draw and author a non-rectangular, rotated bridge detail without writing formulas; a draftsman can change exposed dimensions and receive invariant-preserving geometry with an auditable solve report.

Non-negotiable test gates

- Analytical or automatic-derivative checks for each residual/Jacobian against finite differences away from known singularities.
- Rotation and translation invariance: run every predicate fixture under multiple rigid transforms and compare candidate type/value within model tolerance.
- No mutation test: rejected or inadmissible candidates cannot modify persistent geometry or parameters.
- Constraint diagnosis tests for independent, redundant, conflicting, malformed, and non-convergent graphs.
- Continuity tests: repeated drag/commit cycles do not flip a valid chamfer or produce a topological gap.
- Bridge invariants: changing clear span/cell count preserves declared thicknesses, angles, symmetry, and port alignment.
- Generality proof: an unrelated fixture such as the railing/post system uses the same primitive/constraint/port paths.
- Performance tests at isolated-component and dense-connected-component scales. FreeCAD has recently documented diagnosis costs growing quadratically across many independent sketch components, and reports large gains from partitioning them first [FreeCAD issue 31183](#). Treat component partitioning as a correctness and performance requirement.

Boundaries and risks

No generic CAD engine can learn a user's intended hierarchy from coincidental geometry alone. The system can measure a 350 mm offset but cannot know whether it represents wall thickness, clearance, cover, or a decorative gap. Author confirmation and template reuse are therefore part of correctness, not optional UX.

The initial protocol covers exact line and circular geometry. Splines, ellipses, imported imperfect geometry, dimension styles, and standards-specific civil checks need explicit adapter and residual work. Preserve them in files if necessary, but never claim their relationships are editable until conformance tests exist.

Do not place generative AI in the constraint or solve path. Recent Autodesk work reports that constraint generation remains a design-alignment problem and evaluates generated constraints using solver feedback [Autodesk ICCV 2025](#). If AI is later used, restrict it to ranking or naming candidates. The deterministic predicate engine, admissibility test, author confirmation, and numeric solver remain the source of truth.

Recommendation

Approve the dual-model architecture and GEOM-RP/2 as an internal protocol. Fund Phases 0 through 3 as the first product milestone. That milestone changes the system from axis-aligned formula inference to typed, rotation-invariant constraint candidates and bidirectional dimension editing. Do not begin broad template migration, optional AI ranking, or a production PlaneGCS dependency until those foundations and their fixtures are passing.

Claim to source ledger

ID	Claim supported	Source and access note
S1	Current branch state, file behaviour, test result	Direct local inspection on 7 September 2026 at commit 3c4956a; bun run test result recorded in this report.
S2	Sketch constraints, dimensional editing, and DoF workflow	FreeCAD Sketcher Workbench , current documentation.
S3	PlaneGCS primitive support, temporary constraints, algorithms, WebAssembly wrapper	Salusoft89 Planegcs README , accessed 7 September 2026.
S4	Model-space topology/geometry separation and exchange boundary	ISO 10 303-42 and ISO 10 303-108 , public scope pages, accessed 7 September 2026.
S5	Explicit-constraint and dimension vocabulary	ISO TS 10 303-1788 and ISO TS 10 303-1789 , public scope pages, accessed 7 September 2026.
S6	Requirement keywords for internal protocol	RFC 2119 , March 1997, updated by RFC 8174.
S7	Constraint-generation research and solver verification principle	Autodesk Research ICCV 2025 , accessed 7 September 2026.
S8	Connected-component diagnosis performance evidence	FreeCAD issue 31 183 , open issue, accessed 7 September 2026.
S9	Mathematical and product-design context	Supplied Parametric 2D CAD Geometry Kernel Architecture.pdf, Parametric CAD Engine Architecture Blueprint.pdf, Parametric 2D CAD Kernel Architecture.md, universal-geometric-resolver-protocol.md, 2d-canvas-prd.md, Masterplan.md, and research-cleanup notes. These were read as reference material; claims above that concern the live code were rechecked locally.